CTX

Context Compiler와 표현의 경제학

prompt를 문자열이 아니라 provenance·순서·cache 제약을 가진 실행 표현으로 이해한다

> **모델의 추론 품질은 호출 함수보다 호출 전에 어떤 세계를 어떤 순서로 구성했는지에 좌우된다.**

대상 snapshot
v2.1.88 / 2ca5ddabfed5

설명 구조
조건 · 이론 · 구현 · 반례

제3자 복원 소스에 대한 교육용 분석이다. 코드 관찰, 이론적 비유, 개선 제안을 구분하며 누락 모듈을 추정해 채우지 않는다.

모델은 저장소를 직접 보지 않는다

코드베이스의 어떤 부분이 실제 model world가 되는가?

성립 조건

모델 입력에는 system, tools, messages, attachments, memory, compact summary가 제한된 budget 안에서 합류한다.

이론 렌즈

Knowledge Projection은 canonical source와 consumer-specific compression을 분리한다.

코드의 구체화

QueryEngine이 session context를 모으고 queryContext/API layer가 provider request 형태로 낮춘다.

깨지는 지점

파일이 저장소에 존재해도 선택·순서·변환에서 빠지면 모델에게는 존재하지 않는다.

해석 context construction은 retrieval이 아니라 세계 모델의 선택·변환·직렬화다.

근거와 연결 src/QueryEngine.ts · src/utils/queryContext.ts · artifacts/knowledge/operations/knowledge-projection-and-contextualization.md

compiler 비유는 transformation invariant에서 성립한다

문맥 조립을 컴파일이라고 부를 근거는 무엇인가?

성립 조건

서로 다른 source language가 provider-specific ordered blocks로 변환된다.

이론 렌즈

IR은 여러 front end와 target 사이에서 분석·변환 가능한 공통 표현을 제공한다.

코드의 구체화

내부 message/attachment/tool representation이 API payload와 cache layout으로 lowering된다.

깨지는 지점

형식적 문법·정적 의미·code generation 전체가 없으므로 일반 compiler와 동일시할 수 없다.

해석 비유의 유효 범위는 typed normalization, ordered transformation, invariant preservation, target lowering이다.

근거와 연결 LLVM IR · MLIR · src/services/api/claude.ts

좋은 중간 표현은 출처를 잃지 않는다

> 문자열 배열보다 무엇이 더 필요할까?

필드	보존해야 할 이유	손실 시 실패
kind/role	provider protocol과 pairing	user/tool/system 의미 혼동
provenance	삭제·압축·재생 판단	summary가 원문처럼 권위화
stability class	cache prefix 배치	매 turn cache invalidation
visibility	UI/SDK/model 소비 차이	progress가 reasoning history로 유입
durability	resume 포함 여부	현재 화면과 재개 state 불일치
budget cost	압축/제외 판단	중요 constraint가 뒤늦게 절단

근거와 연결 W3C PROV-O · src/utils/messages.ts

내용과 유래는 서로 다른 정확성 축이다

같은 문장이 어디서 왔는지가 왜 중요한가?

성립 조건

user instruction, tool observation, compact summary, memory는 같은 text라도 다른 신뢰·수명·override 규칙을 가진다.

이론 렌즈

PROV는 Entity, Activity, Agent와 derivation을 분리한다.

코드의 구체화

message kind와 metadata가 projection·resume·provider normalization에서 다른 처리를 받는다.

깨지는 지점

압축된 inference를 원 관찰처럼 취급하면 다음 turn이 잘못된 확신을 누적한다.

해석	context correctness는 text accuracy뿐 아니라 lineage와 transformation honesty를 포함한다.

근거와 연결 PROV-O · artifacts/knowledge/governance/contextual-knowledge-and-repository-bias-control.md

순서는 의미와 비용을 동시에 바꾼다

왜 같은 block 집합의 재정렬이 동등하지 않은가?

instruction precedence, recency, tool/result adjacency, cache prefix가 모두 순서에 의존한다. 따라서 context는 set이 아니라 ordered, typed sequence다. “내용이 같다”는 것은 provider request의 동등성을 보장하지 않는다.

구조적 귀결	정렬 안정성은 performance 최적화가 아니라 semantic/cache compatibility contract가 된다.

근거와 연결 src/tools.ts · src/services/api/claude.ts · Anthropic prompt caching

cache는 data structure의 모양을 architecture로 끌어올린다

왜 tool 순서를 안정화해야 하는가?

성립 조건

Anthropic cache는 tools→system→messages의 prefix 계층을 사용한다.

이론 렌즈

공통 prefix reuse는 동일성 판단이 앞부분의 byte/order 변화에 민감하다는 뜻이다.

코드의 구체화

tool pool ordering과 queryContext의 cache-key prefix가 request 조립과 인접한다.

깨지는 지점

시간·상태를 static prefix에 섞거나 tool list를 흔들면 의미는 같아도 cache가 깨진다.

해석

cache-aware architecture는 안정 content를 앞에, volatile content를 뒤에 두는 change-frequency partitioning이다.

근거와 연결 official prompt caching · src/utils/queryContext.ts · src/tools.ts

tool schema는 모델이 사용할 언어의 문법이다

왜 도구 추가가 prompt 변경이기도 한가?

성립 조건

모델은 tool name, description, input schema를 통해 가능한 행동을 배운다.

이론 렌즈

문법은 표현 가능한 program space를 정의하지만, 실행 권한이나 의미의 참을 보장하지 않는다.

코드의 구체화

Tool은 schema와 prompt description, call, result mapping을 한 계약에 결합한다.

깨지는 지점

schema만 바꾸고 permission·presentation·history를 갱신하지 않으면 같은 이름의 언어가 다른 의미를 갖는다.

해석 tool registry는 API 목록이 아니라 model-facing language environment다.

근거와 연결 src/Tool.ts · src/tools.ts · MCP Tools

TypeScript 타입과 runtime schema는 다른 실패를 막는다

> 컴파일 성공이 model input을 안전하게 만드는가?

경계	보장	보장하지 않음
TypeScript	개발 시 표현 관계	외부 JSON shape
Zod/runtime parse	현재 process의 input shape	path 존재·권한·의도
tool validation	tool-specific semantic precondition	사용자 승인
permission	정책상 실행 가능성	effect 성공·최신 상태
result mapping	model/UI 관찰 형태	durable settlement

여러 검증층은 중복이 아니라 서로 다른 proposition을 판정한다.

근거와 연결 src/services/tools/toolExecution.ts · JSON Schema 2020-12

attachment는 메시지 부록이 아니라 late-bound context다

파일 변화와 hook 결과는 언제 message가 되는가?

attachment는 UI 입력 순간의 text와 다른 시간에 계산되고, 일부는 API normalization에서 user content로 변환된다. 이것은 context가 한 번에 생성되는 값이 아니라 여러 observation source를 합성한 snapshot임을 보여 준다.

학습 포인트	attachment를 읽을 때 content뿐 아니라 생성 시점, source state, model visibility, transcript durability를 함께 추적한다.

근거와 연결 src/utils/attachments.ts · src/utils/processUserInput/processUserInput.ts

memory는 더 긴 prompt가 아니라 별도 수명의 지식이다

세션 message와 memory를 왜 구분해야 하는가?

성립 조건

memory는 turn을 넘어 재사용되며 현재 request보다 오래 살 수 있다.

이론 렌즈

reusable knowledge와 execution context는 수명·owner·compression 경계가 다르다.

코드의 구체화

memdir와 context assembly가 durable knowledge를 현재 query에 투영한다.

깨지는 지점

현재 task의 추측이 memory로 승격되면 future context에 authority drift가 생긴다.

해석 memory quality는 recall 양이 아니라 provenance, applicability, invalidation을 보존하는가로 평가해야 한다.

근거와 연결 src/memdir/memdir.ts · artifacts/knowledge/operations/knowledge-projection-and-contextualization.md

compact는 요약 기능이 아니라 lossy state migration이다

> token을 줄이는 대신 무엇을 잃을 수 있는가?

성립 조건

history가 budget을 넘으면 일부 과거를 summary로 바꾸어 다음 request를 계속해야 한다.

이론 렌즈

lossy transformation은 source snapshot, omitted claims, reason, invalidation을 추적해야 한다.

코드의 구체화

query loop가 budget pressure를 감지하고 compact path를 거쳐 새 context state를 구성한다.

깨지는 지점

permission decision, unresolved error, tool pairing 같은 protected fact가 summary에서 사라지면 future action이 달라진다.

해석	compact의 acceptance는 "짧아졌다"가 아니라 future decision equivalence를 얼마나 보존했는가다.

근거와 연결 src/query.ts · src/services/compact/compact.ts · artifacts/knowledge/operations/knowledge-projection-and-contextualization.md

모든 token은 같은 가치가 아니다

> 무엇을 먼저 압축해야 하는가?

정보 종류	손실 비용	처리 원칙
권한·금지 constraint	unauthorized effect	보호·반복 가능
unresolved failure	같은 실패 재실행	원인·fingerprint 보존
tool identity/pairing	protocol invalid	구조 보존
대화 수사·중복 설명	낮음	요약/제거 후보
raw observation	상황 의존	digest + 필요한 excerpt
derived conclusion	오류 누적	inference 표지 + source 연결

비보상 규칙 많은 저가치 token을 줄였다는 이득이 protected constraint 하나의 손실을 보상하지 못한다.

관찰용 변환이 실행 입력을 오염시키면 안 된다

왜 observer를 위한 clone이 필요한가?

provider fallback이나 observer instrumentation은 같은 request를 다른 목적으로 본다. 관찰을 위해 mutable message를 수정하면 cache identity와 실제 호출 의미까지 변할 수 있다. 그래서 관찰 projection은 execution input과 identity를 분리해야 한다.

근거와 연결 src/query.ts · artifacts/knowledge/operations/knowledge-projection-and-contextualization.md

context budget은 단순 길이 제한이 아니다

> 어떤 경쟁을 조정하는 정책인가?

경쟁	한쪽을 늘릴 때	다른 쪽의 위험
tools vs history	능력 표현 증가	작업 기억 감소
raw evidence vs summary	grounding 증가	token/latency 증가
stable prefix vs personalization	cache reuse 증가	현재성 감소
progress visibility vs reasoning purity	관찰성 증가	noise 유입

따라서 token count 하나로 최적화하지 않고 correctness·cache·latency·omission risk를 vector로 본다.

Context compiler는 golden text보다 metamorphic property로 시험한다

provider request 전체 snapshot만으로 충분한가?

변환	보존되어야 할 property	탐지하려는 실패
tool input 순서 변화 없음	stable prefix hash	비결정적 registry
UI-only progress 추가	model request 불변	projection leakage
compact 실행	protected constraint 유지	lossy omission
observer 활성화	execution payload 불변	instrumentation mutation
resume	tool pairing/terminal 유지	history repair drift

oracle snapshot은 변화를 보여 주고, metamorphic property는 어떤 변화가 허용되지 않는지 설명한다.

명시적 ContextIR이 필요한 이유

새 타입을 만드는 것이 complexity를 늘리는가 줄이는가?

성립 조건

현재 context 규칙은 QueryEngine, query, API, messages, compact에 분산된다.

이론 렌즈

공통 IR은 transformation contract와 analysis seam을 제공한다.

코드의 구체화

source ref, role, visibility, stability, durability, budget cost를 가진 block union을 경계에 둘 수 있다.

깨지는 지점

IR이 provider payload를 그대로 복제하거나 모든 metadata를 끌어안으면 새 god object가 된다.

해석 IR의 가치는 중앙화가 아니라 변환 전후에 반드시 보존할 property를 명시하는 데 있다.

근거와 연결 src/QueryEngine.ts · src/services/api/claude.ts · LLVM IR

실습: cache hit보다 의미 동등성을 먼저 측정한다

> 어떤 실험이 context 설계를 드러내는가?

01	**fixture**	같은 task·tools·history에서 volatile timestamp 하나만 바꾼다.
02	**observation**	request prefix hash, cache read/write token, model-visible blocks를 비교한다.
03	**variant**	tool order, observer, compact, resume를 하나씩 바꾼다.
04	**falsification**	동일해야 할 request가 달라지거나 다른 request가 같아지면 실패다.
05	**design note**	어떤 block field와 owner가 차이를 설명해야 하는지 기록한다.

Context는 모델 앞의 작은 운영체제다

이 권에서 남겨야 할 한 문장은 무엇인가?

> **모델이 보게 되는 세계는 발견되는 것이 아니라, provenance·budget·cache·policy 아래 컴파일된다.**

따라서 context bug는 prompt wording 문제만이 아니다. 잘못된 source 선택, ordering, projection leakage, lossy migration, cache coupling도 모두 같은 제품 결과를 바꾼다.